

Lab Task

Subject: Operating System

Topic: System call and file handling



THE
UNIVERSITY OF
LAHORE

Submitted by:

Mian Daniyal Hassan

Sap ID:

70150006

Dep:

BS CS

Section:

(C)

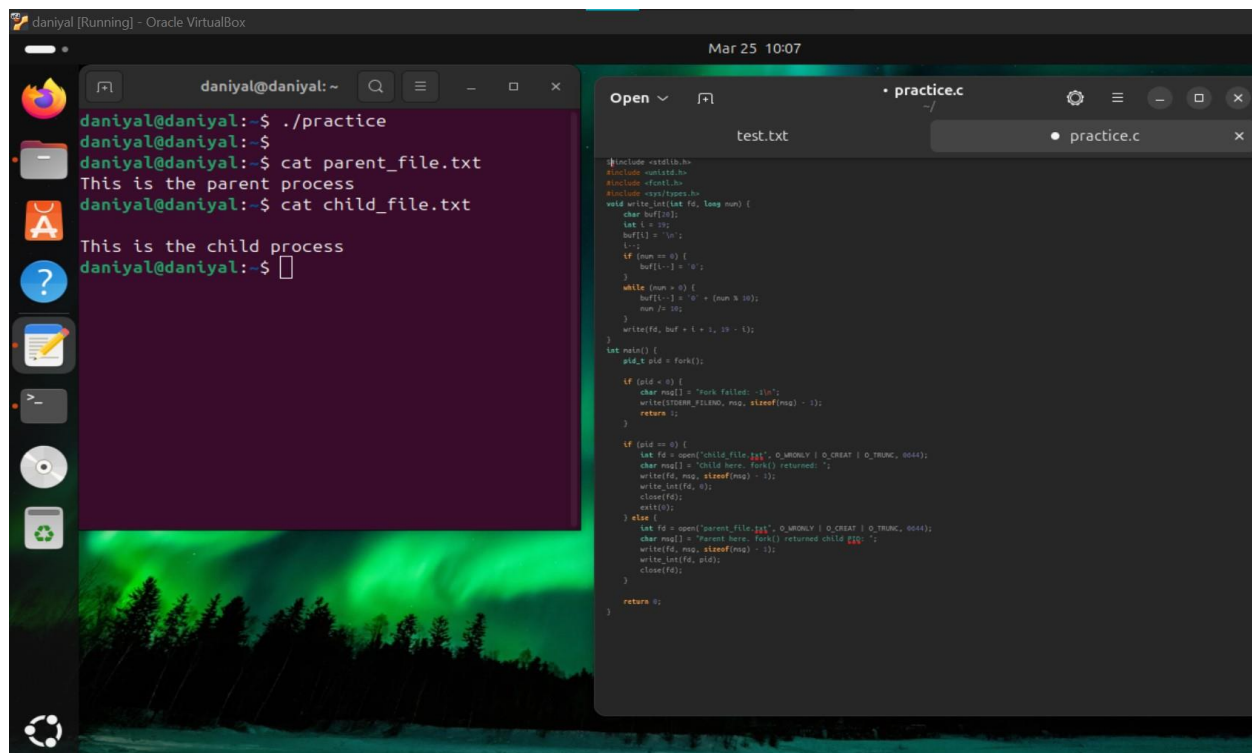
Date:

25th March, 2026

Submitted To:

Ms. Atrooba Nayab

PART # 1



The screenshot shows a VirtualBox window titled 'daniyal [Running] - Oracle VirtualBox'. Inside, there's a terminal window on the left and a code editor on the right. The terminal shows the execution of a program named 'practice', which prints 'This is the parent process' and 'This is the child process'. The code editor shows the source code for 'practice.c', which uses fork() to create a child process and write to two different files.

```
daniyal@daniyal:~$ ./practice
daniyal@daniyal:~$ cat parent_file.txt
This is the parent process
daniyal@daniyal:~$ cat child_file.txt
This is the child process
daniyal@daniyal:~$
```

```
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>

void write_int(int fd, long num) {
    char buf[20];
    int i = 19;
    buf[i] = '\n';
    i--;
    if (num == 0) {
        buf[i--] = '0';
    }
    while (num > 0) {
        buf[i--] = '0' + (num % 10);
        num /= 10;
    }
    write(fd, buf + i + 1, 19 - i);
}

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        char msg[] = "Fork failed: -1\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return 1;
    }

    if (pid == 0) {
        int fd = open("child_file.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        char msg[] = "Child here. Fork() returned: ";
        write(fd, msg, sizeof(msg) - 1);
        write_int(fd, 0);
        close(fd);
    } else {
        int fd = open("parent_file.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        char msg[] = "Parent here. Fork() returned child pid: ";
        write(fd, msg, sizeof(msg) - 1);
        write_int(fd, pid);
        close(fd);
    }

    return 0;
}
```

CODE

```
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>

void write_int(int fd, long num) {
    char buf[20];
    int i = 19;
    buf[i] = '\n';
    i--;
    if (num == 0) {
        buf[i--] = '0';
    }
    while (num > 0) {
        buf[i--] = '0' + (num % 10);
        num /= 10;
    }
    write(fd, buf + i + 1, 19 - i);
}

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        char msg[] = "Fork failed: -1\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return 1;
    }

    if (pid == 0) {
        int fd = open("child_file.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

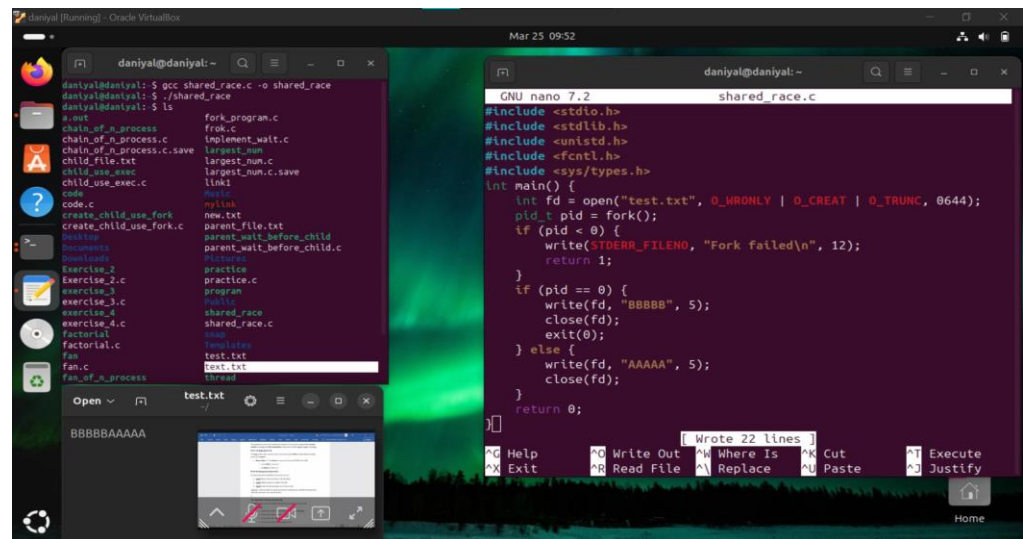
```

char msg[] = "Child here. fork() returned: ";
write(fd, msg, sizeof(msg) - 1);
write_int(fd, 0);
close(fd);
exit(0);
} else {
    int fd = open("parent_file.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    char msg[] = "Parent here. fork() returned child PID: ";
    write(fd, msg, sizeof(msg) - 1);
    write_int(fd, pid);
    close(fd);
}

return 0;
}

```

PART # 2



CODE

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/types.h>

int main() {

    int fd = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    pid_t pid = fork();

    if (pid < 0) {

        write(STDERR_FILENO, "Fork failed\n", 12);

        return 1;

    }

```

```
if (pid == 0) {  
    write(fd, "BBBBB", 5);  
    close(fd);  
    exit(0);  
} else {  
    write(fd, "AAAAA", 5);  
    close(fd);  
}  
return 0;  
}
```